
Sharelock OZmes & Web3son

Implémentation d'une blockchain simplifiée en Oz

Groupe 34

Nzongan Asaph — NOMA : 0667 2400

Boedt Henri-Maximilien — NOMA : 7151 2400



Résumé

L'objectif de ce projet était de reconstruire une version simplifiée de blockchain capable de parcourir une liste de transactions ordonnées par bloc, de valider chacune d'elles, puis d'assembler les blocs résultants dans une chaîne cohérente.

Année académique 2025-2026

1 Implémentation du système

L'implémentation s'articule autour de deux procédures principales : **ExecuteBlockchain**, qui sert de point d'entrée, et **AppendToBlockchain**, qui réalise le travail itératif de validation et d'agrégation. L'état du réseau est représenté sous la forme de trois listes parallèles (**User**, **Balance**, **Nonce**), encapsulées dans des cellules. Ce choix de structure permet de maintenir une correspondance directe entre l'identité d'un utilisateur et ses attributs (solde et nonce) via un index commun.

1.1 Procédure ExecuteBlockchain

Cette procédure initialise les structures de données nécessaires au traitement. Elle crée les cellules **User**, **Balance**, **Nonce** et **Blockchain** à **nil**, puis invoque **ExtractGenesisState**. Cette dernière parcourt le *record* génésis pour peupler les listes initiales, créant une cellule individuelle pour chaque solde et chaque nonce. Une fois l'état initial établi, la procédure délègue le traitement des transactions à **AppendToBlockchain**. Enfin, elle construit **FinalState** et **FinalBlockchain** en inversant les accumulateurs pour garantir un ordre chronologique conforme.

1.2 Procédure AppendToBlockchain

Cette procédure traite la liste des transactions de manière séquentielle et récursive. Pour chaque transaction, elle effectue les étapes suivantes :

- **Validation** : Elle vérifie l'intégrité de la transaction via **ValidateTransaction** (vérification du hash, du nonce et de la suffisance du solde).
- **Mise à jour de l'état** : En cas de succès, elle met à jour les cellules du *sender* (déduction du montant) et du *receiver* (ajout du montant). Si le destinataire n'est pas encore répertorié dans l'état, il est ajouté dynamiquement aux listes parallèles.
- **Agrégation en blocs** : Elle surveille le champ **block_number**. Lorsqu'une transaction présente un numéro de bloc différent de la précédente, elle finalise le bloc actuel en calculant son hash (incluant le hash du bloc précédent) et l'insère dans la chaîne via **AddBlock**.

1.3 Fonction Decode

La fonction **Decode** extrait le message secret en analysant la chaîne de blocs finalisée. Elle parcourt chaque bloc, convertit son hash numérique en une liste de chiffres, puis les traite par paires ($X = 10d_1 + d_2$). Le calcul $X \bmod 37$ détermine l'indice du caractère dans la table Sharelock. Une règle spécifique transforme tout résultat strictement inférieur à 10 en la valeur 36, correspondant au caractère espace. Les caractères ainsi obtenus sont concaténés pour reconstruire la chaîne de texte finale.

2 État final de la blockchain

Solde et dernier nonce de chaque utilisateur

Adr.	Solde	Nonce	Adr.	Solde	Nonce	Adr.	Solde	Nonce
14	70 294 080	8	32	5 141 804	7	60	36 160	3
15	94 650 918	6	37	1 239 534 151	0	66	480 590	2
49	463 119	1	11	97 415	2	73	58 497	4
61	221 043	4	86	918 173	2	87	210 757	1
100	579 432	3						

Représentation de la blockchain finale

n°	previousHash	hash	hashs des transactions
0	0	256 424	22472, 38896, 81865, 61514, 51677
1	256424	153 662	363117, 465569, 68551
2	153662	146 614	751304, 50438, 62480, 128728
3	146614	647 331	291119, 170096, 24257, 15242
4	647331	102 336	247790, 139871, 67340
5	102336	272 434	92750, 47822, 23514, 6007
6	272434	551 027	24505, 181165, 72917
7	551027	297 173	479975, 16061, 78363, 133002, 38738
8	297173	552 873	87525, 106662, 61505
9	552873	485 117	281209, 582818, 68208
10	485117	552 350	32842, 11083, 23298
11	552350	732 472	16992, 42982, 120137

Phrase secrète

L'application de la procédure Decode à la blockchain finale produit :

prof peter van royiarty is behind oz

3 Extension 1 : Denylist

Le principe de cette extension est de bannir tout utilisateur ayant émis au moins trois transactions au sein d'un même bloc, valides ou non, et de le maintenir banni pour le reste de la chaîne.

Choix d'implémentation : La structure d'état est étendue avec une quatrième liste parallèle `CountTransac`, où chaque entrée est un *record* `ct(last_block:<cell>, count:<cell>)` associant à un utilisateur le numéro du dernier bloc dans lequel il a tenté d'émettre ainsi qu'un compteur du nombre de tentatives dans ce bloc. Lorsqu'une nouvelle transaction est rencontrée pour un utilisateur, deux cas se présentent : si le `block_number` courant diffère du `last_block` mémorisé *et* que le compteur est encore strictement inférieur à 3, on remet ce compteur à zéro avant de l'incrémenter (l'utilisateur entre dans un nouveau bloc « propre ») ; sinon on incrémente simplement. La condition `@(C.count) =< 3` est ajoutée à `ValidTransaction` : dès qu'un compteur dépasse cette borne, l'utilisateur est de facto figé sur la denylist, son compteur n'étant plus jamais remis à zéro pour les blocs ultérieurs.

État final

Adr.	Solde	Nonce	Adr.	Solde	Nonce	Adr.	Solde	Nonce
14	70 226 791	8	32	6 107 292	3	60	36 160	3
15	94 043 702	6	37	1 239 534 151	0	66	457 435	2
49	463 119	1	11	97 415	2	73	81 652	3
61	221 043	4	86	918 173	2	87	210 757	1
100	288 449	3						

Liste des blocs

n°	previousHash	hash	hashs des transactions
0	0	256 424	22472, 38896, 81865, 61514, 51677
1	256424	153 662	363117, 465569, 68551
2	153662	146 614	751304, 50438, 62480, 128728
3	146614	356 212	170096, 24257, 15242
4	356212	743 877	247790, 139871
5	743877	913 975	92750, 47822, 23514, 6007
6	913975	168 063	181165, 72917
7	168063	914 209	479975, 16061, 78363, 133002, 38738
8	914209	169 909	87525, 106662, 61505
9	169909	519 335	281209, 68208
10	519335	563 270	32842, 11083
11	563270	743 392	16992, 42982, 120137

Phrase secrète obtenue :

prof petezpc h g qh gp ejzjwx xi

Comme prévu par l'énoncé, la denylist filtre certaines transactions, ce qui modifie les hashs des blocs concernés et donc la phrase décodée, qui n'a effectivement plus de sens littéraire.

4 Extension 2 : L'effort n'est pas gratuit

La seconde extension intègre le coût de l'effort dans la validation : chaque unité d'effort coûte une unité monétaire à l'envoyeur, ce qui se traduit par une déduction supplémentaire de $\{\text{Effort } T\}$ sur son solde, en plus de la **value** transférée au receveur.

Choix d'implémentation : Deux modifications minimales suffisent par rapport au module de base. D'une part, dans `ValidateTransaction`, la condition de solde devient $T.\text{value} + \{\text{Effort } T\} \leq @(\text{Balance}.1)$: la transaction est refusée dès que l'envoyeur ne peut payer *à la fois* le montant et le coût de l'effort. D'autre part, dans `UpdateStateForSender`, la mise à jour du solde devient $B := @B - T.\text{value} - \{\text{Effort } T\}$. Le receveur n'est pas affecté par l'effort : il reçoit toujours exactement la **value** indiquée. Le reste de la logique (clôture des blocs, plafond de 300 unités d'effort par bloc, calcul des hashes) reste identique au module de base, ce qui permet une comparaison directe des deux exécutions.

État final

Adr.	Solde	Nonce	Adr.	Solde	Nonce	Adr.	Solde	Nonce
14	70 293 640	8	32	5 141 523	7	60	36 035	3
15	94 650 668	6	37	1 239 534 151	0	66	480 496	2
49	463 088	1	11	97 353	2	73	58 341	4
61	220 919	4	86	918 127	2	87	210 726	1
100	579 275	3						

Liste des blocs

Avec le jeu de transactions fourni, tous les envoyeurs disposent d'un solde largement suffisant pour absorber le coût supplémentaire de l'effort : aucune transaction n'est rejetée en plus par rapport au cas de base, et la chaîne de hashes reste donc strictement identique à celle de la section 2. Seuls les soldes finaux diffèrent (montants légèrement plus faibles, voir tableau ci-dessus).

n°	previousHash	hash	hashs des transactions
0	0	256 424	22472, 38896, 81865, 61514, 51677
1	256424	153 662	363117, 465569, 68551
2	153662	146 614	751304, 50438, 62480, 128728
3	146614	647 331	291119, 170096, 24257, 15242
4	647331	102 336	247790, 139871, 67340
5	102336	272 434	92750, 47822, 23514, 6007
6	272434	551 027	24505, 181165, 72917
7	551027	297 173	479975, 16061, 78363, 133002, 38738
8	297173	552 873	87525, 106662, 61505
9	552873	485 117	281209, 582818, 68208
10	485117	552 350	32842, 11083, 23298
11	552350	732 472	16992, 42982, 120137

Phrase secrète obtenue :

prof peter van royiarty is behind oz

La phrase est ici identique à celle du cas de base, conséquence directe du fait que les hashes des blocs ne changent pas avec ce jeu de transactions.

Utilisation de l'IA : Conformément aux consignes, l'IA a été utilisée uniquement pour la relecture orthographique et la mise en forme du présent rapport. Le fond du rapport et le code Oz ont été rédigés et écrits par les auteurs.